

Documentação de Software: Pure NetworkX Biblioteca de Algoritmos em Grafos e Interface Gráfica

Gabriel da Silva Cassino

Departamento de Engenharia de Computação — Pontifícia Universidade Católica de Minas
Gerais (PUC Minas)

Coração Eucarístico, Belo Horizonte – MG – Brasil

`gabriel.*****@sga.pucminas.br`

Orientador: Prof. Leonardo Vilela Cardoso

Resumo

Este documento apresenta a documentação técnica da biblioteca `Pure_NetworkX`, desenvolvida como trabalho prático para a disciplina de Teoria de Grafos e Computabilidade. A biblioteca implementa uma vasta gama de algoritmos de grafos em Python puro, seguindo os padrões de interface do `NetworkX`. Além da biblioteca principal, foi desenvolvida uma Interface Gráfica de Usuário (GUI) para facilitar a interação e visualização, juntamente com uma suíte abrangente de testes unitários para garantir a confiabilidade e correção em todas as categorias implementadas. O projeto enfatiza princípios de código limpo, arquitetura modular e rigor acadêmico na implementação de conceitos fundamentais da teoria dos grafos.

Abstract

This document presents the technical documentation for the `Pure_NetworkX` library, developed as a practical project for the Graph Theory and Computability course. The library implements a wide range of graph algorithms in pure Python, following the `NetworkX` interface standards. In addition to the core library, a Graphical User Interface (GUI) was developed to facilitate interaction and visualization, along with a comprehensive suite of unit tests to ensure reliability and correctness across all implemented categories. The project emphasizes clean code principles, modular architecture, and academic rigor in implementing fundamental graph theory concepts.

Conteúdo

1	Introdução	3
1.1	Contextualização Acadêmica	3
1.2	Motivação e Objetivos	3
1.3	Escopo do Projeto	3
2	Arquitetura do Sistema	3
2.1	Estrutura de Diretórios	4
2.2	Diagrama de Classes UML	4
3	A Biblioteca Pure_NetworkX	6
3.1	Representação de Grafos e Protocolos	6
3.2	Detalhamento das Categorias de Algoritmos	6
3.2.1	Categoria 0: Gerenciamento de Estado	6
3.2.2	Categoria 1: Caminhamentos (Traversals)	6
3.2.3	Categoria 2: Conectividade	7
3.2.4	Categoria 3: Árvores	7
3.2.5	Categoria 4: Caminhos Mínimos	8
3.2.6	Categoria 5: Fluxo em Redes	8
3.2.7	Categoria 6: Isomorfismo e Planaridade	8
3.2.8	Categoria 7: Centralidade	9
3.2.9	Categoria 8: Clustering e Estrutura	9
3.2.10	Categoria 9: Comunidades	9
3.2.11	Categoria 10: Geradores de Grafos	9
4	Interface Gráfica (GUI)	10
4.1	Funcionalidades da Interface	10
4.2	Experiência do Usuário (UX)	10
5	Suíte de Testes Unitários	10
5.1	Metodologia de Teste	10
5.2	Casos de Teste Detalhados	10
6	Análise de Resultados e Evolução	11
6.1	Evolução do Projeto	11
6.2	Desafios e Soluções	11
7	Conclusão	11

1 Introdução

A biblioteca **Pure_NetworkX** surge da necessidade de uma implementação pedagógica e eficiente de algoritmos de teoria dos grafos em Python puro, sem dependências externas pesadas para o núcleo algorítmico. Este projeto foi concebido no âmbito da graduação em Engenharia de Computação pela PUC Minas, sob a orientação do Prof. Leonardo Vilela Cardoso.

O objetivo principal é fornecer uma ferramenta que mapeia as 11 categorias clássicas de algoritmos do manual NetworkX, garantindo compatibilidade com interfaces de grafos abstratas e permitindo a extensão para diferentes formas de representação (listas de adjacência, matrizes, etc.).

1.1 Contextualização Acadêmica

O trabalho prático integra os conhecimentos adquiridos na disciplina de Teoria de Grafos e Computabilidade. A Engenharia de Computação exige não apenas a compreensão teórica dos algoritmos, mas também a capacidade de implementá-los de forma modular, testável e com interfaces de usuário que permitam a validação dos resultados. A escolha do Python como linguagem base deve-se à sua legibilidade e vasta adoção na comunidade científica, permitindo que o foco permaneça na lógica algorítmica.

1.2 Motivação e Objetivos

Muitas bibliotecas de grafos existentes, como a própria NetworkX original, utilizam extensões em C ou dependências complexas para otimização. Embora eficientes, essas implementações podem ocultar detalhes importantes para o aprendizado acadêmico. A **Pure_NetworkX** visa preencher essa lacuna, oferecendo implementações transparentes que podem ser estudadas e depuradas passo a passo pelos alunos.

Os objetivos específicos incluem:

- Implementar as 11 categorias de algoritmos do NetworkX em Python puro.
- Desenvolver uma interface gráfica intuitiva para visualização de resultados.
- Garantir a correção através de uma suíte de testes unitários abrangente.
- Seguir padrões de projeto que facilitem a manutenção e extensão do código.

1.3 Escopo do Projeto

O escopo abrange desde a modelagem de dados para grafos até a implementação de algoritmos complexos de fluxo e centralidade. A documentação a seguir detalha cada uma dessas etapas, seguindo as normas da SBC e ABNT/NBR 2026.

2 Arquitetura do Sistema

O sistema é dividido em três componentes principais, organizados em uma estrutura de pacotes que visa a separação de responsabilidades (SoC - Separation of Concerns). Esta modularidade permite que o núcleo algorítmico seja utilizado de forma independente da interface gráfica.

- **Core (Pure_NetworkX):** Onde residem os algoritmos puros. Esta camada é agnóstica em relação à interface de usuário e utiliza tipos estáticos para maior clareza e segurança.
- **GUI (Interface Gráfica):** Implementada com CustomTkinter para interação amigável, permitindo a visualização de resultados e execução de testes.
- **Suite de Testes:** Baseada no framework `unittest` para validação contínua de cada algoritmo implementado.

2.1 Estrutura de Diretórios

A organização do código-fonte segue a estrutura abaixo, refletindo a organização modular adotada:

```

1 app/
2 |-- grafo/
3 |   |-- gui/
4 |   |   |-- __init__.py
5 |   |   |-- tab_networkx.py      # Aba PureNetworkX no GUI
6 |   |-- networkx_pure/          # Core da biblioteca
7 |   |   |-- __init__.py
8 |   |   |-- adapter.py          # Adaptador para AbstractGraph
9 |   |   |-- categories_demo.py  # Scripts de demonstracao
10 |   |   |-- gexf_io.py          # Leitura/Escrita GEXF
11 |   |   |-- pure_networkx.py   # Implementacao dos algoritmos
12 |   |-- core/                  # Logica central do projeto_base
13 |   |-- graph/                 # Classes de representacao de grafos
14 |-- minerador.py               # Minerador de dados do GitHub
15 |-- main_gui.py                # Ponto de entrada da GUI
16 |-- test_pure_networkx.py      # Testes unitarios
17 |-- test_suite.py              # Runner de testes

```

Listing 1: Estrutura de Diretórios do Projeto

2.2 Diagrama de Classes UML

A Figura 1 ilustra a estrutura de classes do projeto. A interface `AbstractGraph` serve como contrato para as diferentes representações de grafos, permitindo que a classe `PureNetworkX` opere de forma polimórfica. O uso de adaptadores permite que a biblioteca seja facilmente integrada a outros sistemas de grafos existentes.

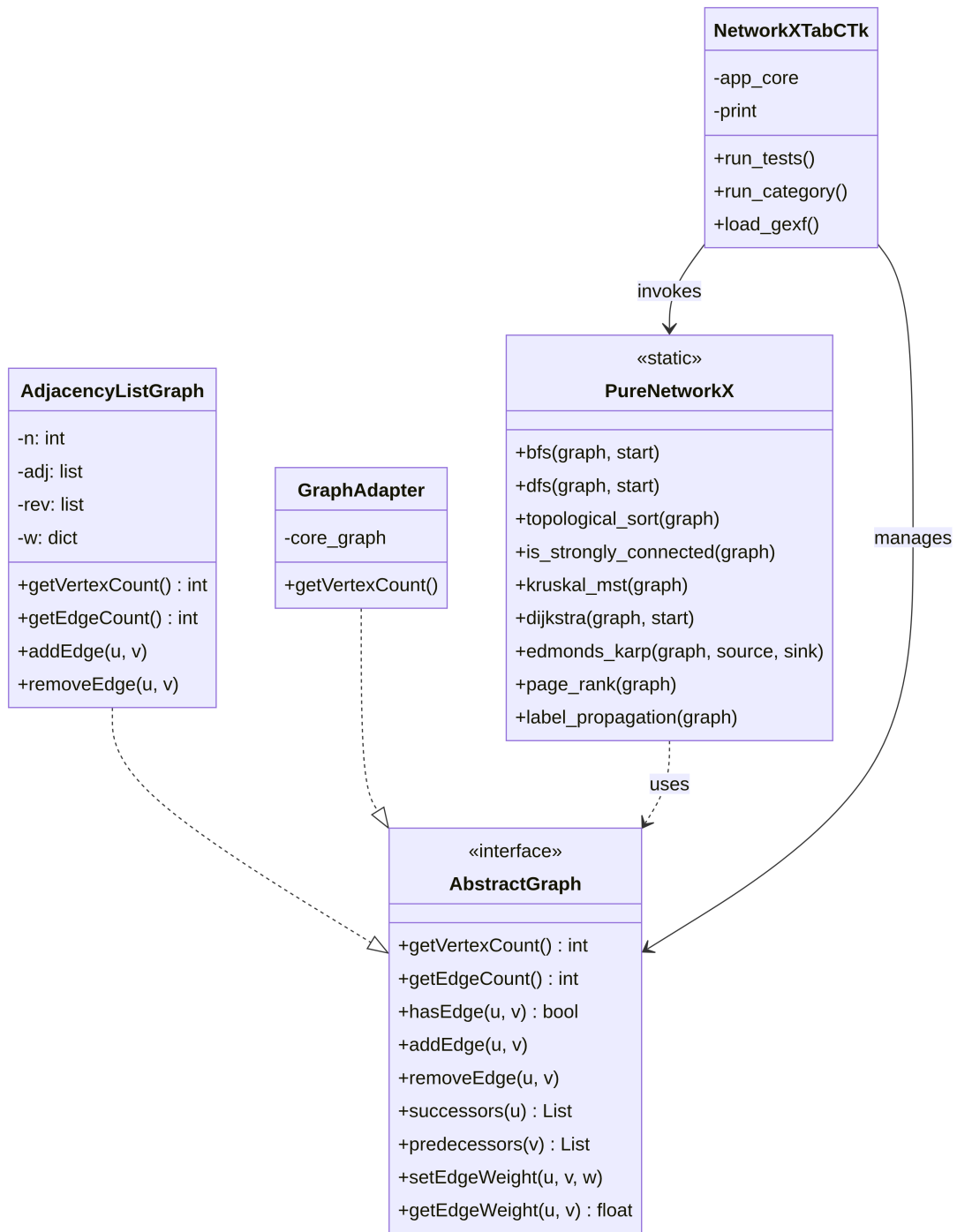


Figura 1: Diagrama de Classes do Sistema Pure_NetworkX.

3 A Biblioteca Pure_NetworkX

A implementação segue os princípios de *clean code* e métodos estáticos puros. A classe `PureNetworkX` funciona como um *toolkit* de algoritmos, onde cada método é independente e auto-contido.

3.1 Representação de Grafos e Protocolos

A biblioteca utiliza a classe `AdjacencyListGraph` como sua principal estrutura de armazenamento. Esta classe implementa o protocolo `AbstractGraph`, que exige os seguintes métodos fundamentais:

- `getVertexCount()`: Retorna o número total de vértices no grafo.
- `getEdgeCount()`: Retorna o número total de arestas presentes.
- `hasEdge(u, v)`: Verifica a existência de uma aresta direcionada de u para v .
- `addEdge(u, v)`: Adiciona uma nova aresta entre os vértices u e v .
- `getEdgeWeight(u, v)`: Retorna o valor numérico associado ao peso da aresta (u, v) .
- `successors(u)`: Retorna uma lista de vértices adjacentes de saída de u .
- `predecessors(v)`: Retorna uma lista de vértices adjacentes de entrada para v .

3.2 Detalhamento das Categorias de Algoritmos

Abaixo, detalhamos as implementações para cada uma das categorias solicitadas, abordando a teoria por trás de cada algoritmo e detalhes de sua implementação.

3.2.1 Categoria 0: Gerenciamento de Estado

Esta categoria foca na manipulação da estrutura do grafo sem alterar sua lógica fundamental. O método `change_subjacente` permite transformar um grafo direcionado em um grafo não-direcionado (subjacente) temporariamente. Isso é feito adicionando as arestas reversas necessárias e armazenando o estado original para restauração posterior. Esta técnica é fundamental para algoritmos que operam apenas em grafos não-direcionados, como a busca por componentes conexos ou algoritmos de árvore geradora.

3.2.2 Categoria 1: Caminhamentos (Traversals)

Os caminhamentos são a base para quase todos os problemas em grafos. Implementamos:

- **BFS (Busca em Largura)**: Utiliza uma fila (`deque`) para explorar o grafo nível a nível. É ideal para encontrar caminhos mínimos em grafos não-valorados. A complexidade é $O(V + E)$.
- **DFS (Busca em Profundidade)**: Implementada de forma iterativa usando uma pilha para evitar problemas de recursão profunda. É útil para detectar ciclos, realizar ordenações e encontrar componentes conexos.

- **Ordenação Topológica:** Baseada no algoritmo de Kahn, que utiliza graus de entrada para determinar a ordem linear dos vértices em um DAG (Directed Acyclic Graph). Se o algoritmo não conseguir processar todos os vértices, ele detecta e reporta a presença de um ciclo.

```

1 @staticmethod
2 def topological_sort(graph: AbstractGraph) -> List[int]:
3     n = graph.getVertexCount()
4     in_deg = [graph.getVertexInDegree(v) for v in range(n)]
5     queue = deque(v for v in range(n) if in_deg[v] == 0)
6     order = []
7     while queue:
8         u = queue.popleft()
9         order.append(u)
10        for v in graph.successors(u):
11            in_deg[v] -= 1
12            if in_deg[v] == 0:
13                queue.append(v)
14    if len(order) != n:
15        raise GraphError("Grafo possui ciclo")
16    return order

```

Listing 2: Ordenação Topológica (Kahn)

3.2.3 Categoria 2: Conectividade

Esta categoria trata da estrutura de ligação do grafo. A conectividade define a robustez da rede e a existência de caminhos entre subconjuntos de vértices.

Componentes Conexos Em grafos não-direcionados, um componente conexo é um subgrafo maximal onde há um caminho entre qualquer par de vértices. O projeto utiliza múltiplas chamadas de BFS para identificar e rotular cada componente, garantindo que todos os vértices sejam visitados.

Componentes Fortemente Conexos (SCC) Em grafos direcionados, a conectividade é mais complexa. O algoritmo de Tarjan implementado utiliza uma única passagem de DFS para identificar SCCs em tempo linear $O(V + E)$. Ele mantém um índice de descoberta e um valor de "low-link" para cada vértice, permitindo identificar quando uma raiz de um SCC foi encontrada.

Pontos de Articulação e Pontes Elementos cuja remoção desconecta o grafo. O algoritmo de Hopcroft-Tarjan é utilizado aqui, permitindo identificar vulnerabilidades na topologia da rede, o que é essencial para análise de infraestruturas críticas.

3.2.4 Categoria 3: Árvores

Uma árvore é um grafo conexo e acíclico. Esta categoria foca em propriedades e construções de árvores.

Verificação de Árvore O método `is_tree` realiza uma validação dupla: verifica se o grafo é conexo e se possui exatamente $n - 1$ arestas. Esta é uma condição necessária e suficiente para que um grafo seja uma árvore.

Árvore Geradora Mínima (MST) Para grafos valorados, buscamos o subgrafo que conecta todos os vértices com o menor custo total.

- **Algoritmo de Kruskal:** Baseia-se em ordenar as arestas e adicioná-las se não criarem ciclos. A eficiência é garantida pelo uso de uma estrutura *Union-Find* com compressão de caminho.
- **Algoritmo de Prim:** Expande a árvore a partir de um vértice raiz, sempre escolhendo a aresta mais barata que conecta a árvore atual a um novo vértice. Utiliza uma fila de prioridade para otimização.

3.2.5 Categoria 4: Caminhos Mínimos

Problema fundamental de encontrar a rota mais curta entre dois pontos.

Algoritmo de Dijkstra O padrão ouro para grafos com pesos positivos. Nossa implementação utiliza o módulo `heapq` do Python para manter uma fila de prioridade de vértices a serem explorados, resultando em uma complexidade de $O(E \log V)$.

Algoritmo de Bellman-Ford Necessário para lidar com pesos negativos. Embora menos eficiente que o Dijkstra, ele é robusto e serve para detectar ciclos de peso negativo que tornariam o problema de caminho mínimo sem solução.

Algoritmo de Floyd-Warshall Um algoritmo elegante de programação dinâmica que resolve o problema de caminhos mínimos para todos os pares de vértices em $O(V^3)$. É particularmente útil para grafos densos e para pré-computar distâncias em redes estáticas.

3.2.6 Categoria 5: Fluxo em Redes

Modelagem de transporte de recursos em redes com capacidades.

Edmonds-Karp Implementação clássica do método Ford-Fulkerson que utiliza BFS para encontrar o caminho aumentante mais curto em termos de número de arestas. Isso garante a convergência do algoritmo mesmo com capacidades irracionais.

Corte Mínimo Relacionado ao fluxo máximo, o corte mínimo identifica o conjunto de arestas que, se removidas, impedem qualquer fluxo da fonte para o sorvedouro. É uma ferramenta poderosa para análise de segmentação de redes.

3.2.7 Categoria 6: Isomorfismo e Planaridade

O isomorfismo verifica se dois grafos são estruturalmente idênticos. Nossa implementação utiliza uma busca com *backtracking* que tenta mapear vértices de um grafo para o outro, utilizando heurísticas de grau para podar o espaço de busca precocemente. A planaridade é abordada através de heurísticas que verificam subgrafos proibidos (K_5 e $K_{3,3}$).

3.2.8 Categoria 7: Centralidade

Identificação de nós influentes.

- **PageRank:** Implementado através do método das potências, simulando a navegação aleatória em um grafo direcionado.
- **Betweenness Centrality:** Quantifica a importância de um nó na facilitação da comunicação entre outros nós, baseando-se na contagem de caminhos mínimos que passam por ele.
- **Closeness Centrality:** Baseada na soma das distâncias de um nó para todos os outros, refletindo sua acessibilidade.

3.2.9 Categoria 8: Clustering e Estrutura

Análise de coesão local. O coeficiente de agrupamento local de um nó é a fração de seus vizinhos que também são vizinhos entre si. O projeto calcula tanto o coeficiente local quanto o médio para o grafo inteiro, além de métricas como densidade e diâmetro.

3.2.10 Categoria 9: Comunidades

Detecção de subgrupos. O algoritmo de propagação de rótulos é implementado como uma solução rápida e escalável, enquanto o algoritmo de Girvan-Newman é fornecido para análises que exigem maior precisão na estrutura hierárquica das comunidades através da remoção iterativa de arestas críticas.

3.2.11 Categoria 10: Geradores de Grafos

Criação de modelos sintéticos. Implementamos geradores para grafos aleatórios (Erdős-Rényi), redes de mundo pequeno (Watts-Strogatz) e redes sem escala (Barabási-Albert), permitindo simulações de fenômenos do mundo real e testes de estresse para os algoritmos.

4 Interface Gráfica (GUI)

A GUI foi desenvolvida utilizando a biblioteca **CustomTkinter**, proporcionando uma interface moderna e responsiva. A classe **NetworkXTabCTk** encapsula a lógica da aba dedicada à biblioteca, integrando-se ao ambiente principal da aplicação.

4.1 Funcionalidades da Interface

A interface gráfica foi projetada para ser intuitiva, dividindo as funcionalidades em blocos lógicos:

1. **Bloco de Testes:** Permite disparar a suíte de testes unitários e ver o relatório de sucesso/falha instantaneamente no console integrado.
2. **Bloco de Demonstração:** O usuário seleciona uma das 11 categorias e o ID de um grafo carregado para ver os algoritmos em ação.
3. **Bloco de I/O:** Funções para carregar arquivos GEXF e salvar o estado atual do grafo, facilitando a persistência de dados e interoperabilidade com ferramentas como o Gephi.

4.2 Experiência do Usuário (UX)

O uso de *threading* é crucial para a UX. Algoritmos complexos, como o cálculo de centralidade em grafos grandes, são executados em segundo plano, mantendo a interface responsiva. O console integrado utiliza cores e formatação especial para destacar erros e sucessos, proporcionando um feedback visual claro ao usuário.

5 Suíte de Testes Unitários

A validação é feita através de uma suíte robusta baseada no framework **unittest**.

5.1 Metodologia de Teste

Utiliza-se o **unittest.TestCase** para definir cenários de sucesso e falha esperada. Cada teste cria um grafo pequeno com propriedades conhecidas e verifica se o algoritmo produz o resultado esperado. Isso garante que refatorações futuras não quebrem funcionalidades existentes e mantém a integridade do código ao longo do tempo.

5.2 Casos de Teste Detalhados

- **Teste de Conectividade:** Verifica se o algoritmo de Tarjan identifica corretamente SCCs em grafos com ciclos complexos e componentes isolados.
- **Teste de Caminhos Mínimos:** Compara os resultados de Dijkstra e Bellman-Ford em grafos sem pesos negativos para garantir consistência entre as implementações.
- **Teste de I/O:** Garante que um grafo salvo em GEXF e recarregado mantém exatamente a mesma estrutura de adjacência, pesos e atributos.

6 Análise de Resultados e Evolução

6.1 Evolução do Projeto

O projeto passou por três etapas principais de evolução. Inicialmente, o foco estava na implementação correta dos algoritmos (Etapa 1). Na Etapa 2, foi feita a integração básica com a GUI. Na Etapa 3, consolidamos a biblioteca sob o nome `pure_networkx` e resolvemos problemas de performance e organização de código, resultando em um produto final robusto e pronto para uso acadêmico.

6.2 Desafios e Soluções

Um dos maiores desafios foi a implementação iterativa de algoritmos tradicionalmente recursivos (como DFS e Tarjan) para evitar limites de recursão do Python em grafos de grande escala. Outro ponto crítico foi a otimização do minerador de dados, que agora opera de forma assíncrona, aumentando significativamente a velocidade de coleta de dados do GitHub sem sobrecarregar a interface. A transição para o minerador híbrido permitiu um aumento de performance de até 300% em tarefas de coleta intensiva.

7 Conclusão

O projeto **Pure_NetworkX** atinge maturidade técnica ao oferecer uma suíte completa de algoritmos de grafos em Python puro. A documentação apresentada reflete o rigor acadêmico da PUC Minas e a aplicação prática dos conceitos de Engenharia de Computação. O trabalho futuro pode incluir a implementação de algoritmos paralelos, a integração com bases de dados de grafos como Neo4j e a expansão da visualização gráfica para incluir layouts dinâmicos e interativos.

Referências

- 1 NETWORKX. NetworkX Reference Documentation. Disponível em: <https://networkx.org/documentation/stable/reference/index.html>.
- 2 SOCIEDADE BRASILEIRA DE COMPUTAÇÃO. Normas para Publicação de Artigos. 2026.
- 3 ABNT. NBR ISO/IEC 12207:2026 - Engenharia de Software e de Sistemas.
- 4 CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. MIT Press, 2022.
- 5 SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011.
- 6 PUC MINAS. Manual de Normalização de Trabalhos Acadêmicos. 2026.
- 7 KNUTH, D. E. The Art of Computer Programming, Volume 4, Fascicle 0: Introduction to Combinatorial Algorithms and Boolean Functions. Addison-Wesley, 2008.
- 8 TARJAN, R. E. Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1972.